

## Problem 3

An anisotropic Poisson problem in a two-dimensional domain  $\Omega$  is given by the strong form

$$\begin{aligned} -\nabla \cdot (\mathbf{A} \nabla u) &= f & \text{in } \Omega, \\ u &= u_0 & \text{on } \Gamma, \end{aligned}$$

where the conductivity tensor  $\mathbf{A}(\mathbf{x}) \in \mathbb{R}^{2 \times 2}$  is assumed to be symmetric and positive definite for all  $\mathbf{x}$ ,  $f(\mathbf{x})$  is a given distributed source, and  $u_0(\mathbf{x})$  is the source on the boundary  $\Gamma$ .

Choose  $\Omega$  to be a disc with radius 1 around the origin.

Take the source terms to be

$$f = \exp(-100(x^2 + y^2)) \quad \text{and} \quad u_0 = 0.$$

Use conductivity tensors  $\mathbf{A}(\mathbf{x})$  given by

$$\mathbf{A}_1 = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix} \quad \text{and} \quad \mathbf{A}_2 = \begin{pmatrix} 1 & -5 \\ -5 & 100 \end{pmatrix}$$

## Import FEniCS and define the mesh

```
In [2]: import dolfin as dl
import ufl

# Enable plotting inside the notebook
import matplotlib.pyplot as plt
%matplotlib inline

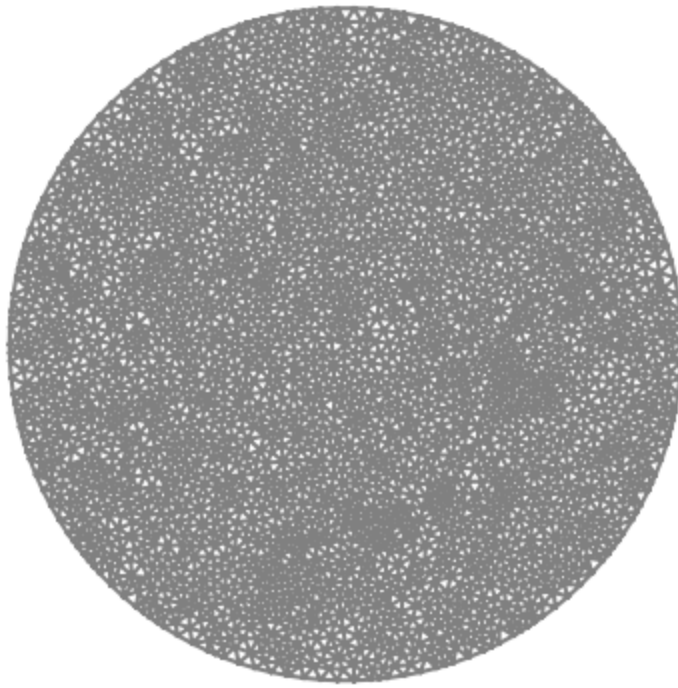
from hippylib import nb

import logging
logging.getLogger('FFC').setLevel(logging.WARNING)
logging.getLogger('UFL').setLevel(logging.WARNING)
dl.set_log_active(False)

mesh = dl.Mesh("circle.xml")

nb.plot(mesh)
```

```
Out[2]: [<matplotlib.lines.Line2D at 0x7c32030340a0>,
<matplotlib.lines.Line2D at 0x7c3203035510>]
```



## Define source terms and conductivity tensors

```
In [4]: f = dl.Expression("std::exp(-100.*(x[0]*x[0] + x[1]*x[1]))", degree=5)
u_0 = dl.Constant(0.)

A1 = dl.Constant(((10.0, 0.0),(0.0, 10.0)))
A2 = dl.Constant(((1.0, -5.0),(-5.0, 100.0)))
```

```
In [5]: d = 2 # quadratic elements
Vh = dl.FunctionSpace(mesh, "Lagrange", d)
```

## Dirichlet BC

```
In [25]: # Compute an approximate radius from the mesh
r = max(dl.sqrt(x[0]**2 + x[1]**2) for x in mesh.coordinates())
def boundary_d(x, on_boundary):
    return (x[0]**2 + x[1]**2 - r**2 < dl.DOLFIN_EPS) and on_boundary

u_d = dl.Constant(0.)
bcs = [dl.DirichletBC(Vh, u_d, boundary_d)]
```

```
In [30]: uh = dl.TrialFunction(Vh)
vh = dl.TestFunction(Vh)
f = dl.Expression("exp(-100*(pow(x[0],2) + pow(x[1],2)))", degree = d+2)
L = f*vh*uf1.dx
```

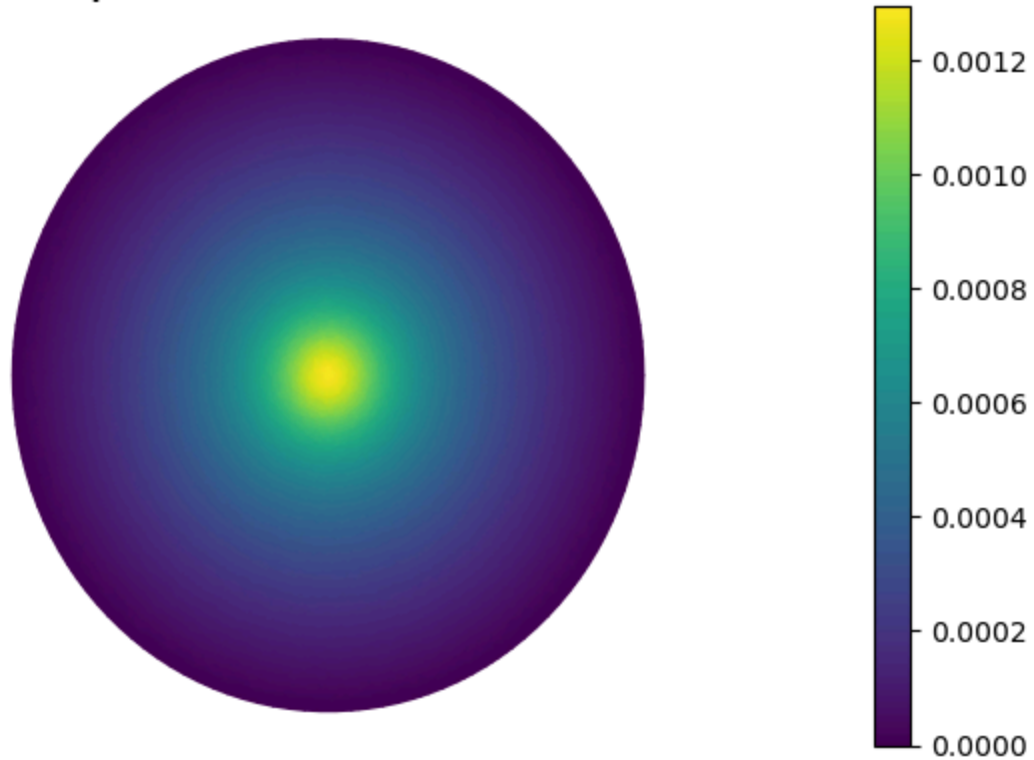
```
In [36]: def solve(a, title):
    A, b = dl.assemble_system(a, L, bcs)
```

```
uh = dl.Function(Vh)
dl.solve(A, uh.vector(), b)

nb.plot(uh, mytitle=title)
plt.show()
```

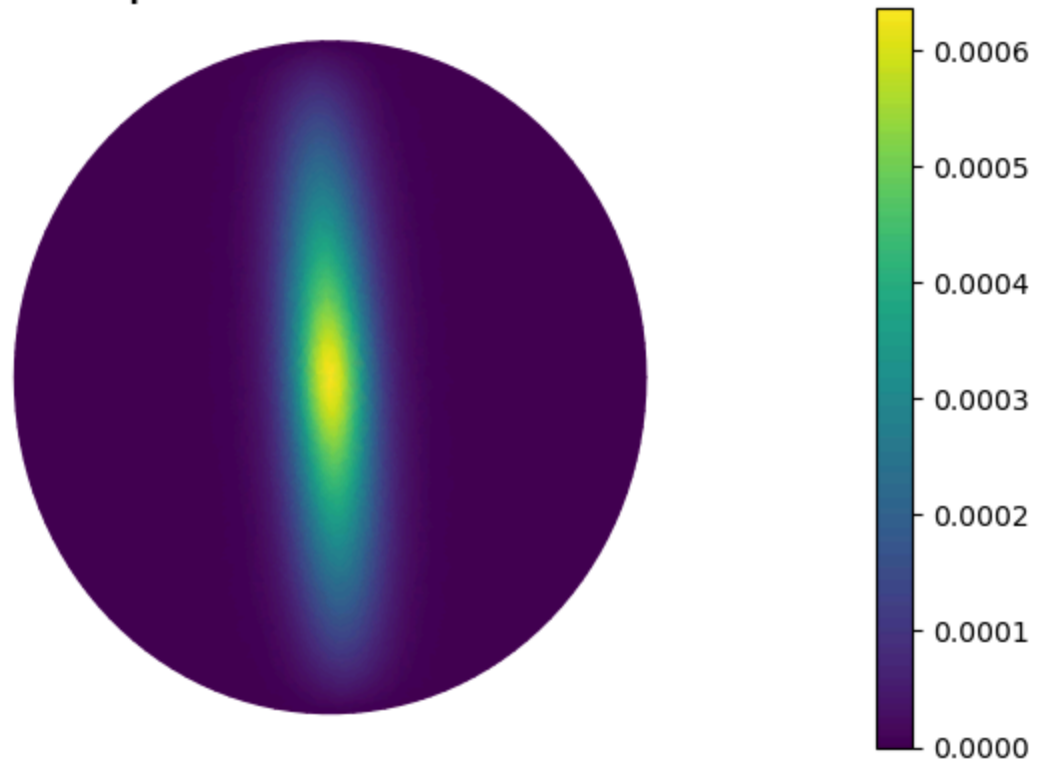
```
In [37]: a = ufl.inner(A1*ufl.grad(uh), ufl.grad(vh))*ufl.dx
solve(a, "Isotropic Poisson FE Solution")
```

## Isotropic Poisson FE Solution



```
In [38]: a = ufl.inner(A2*ufl.grad(uh), ufl.grad(vh))*ufl.dx
solve(a, "Anisotropic Poisson FE Solution")
```

# Anisotropic Poisson FE Solution



Conductivity tensor  $A_1$  is isotropic (1st picture) so results in equal conductivity in all directions, radially. So, the solution is radially symmetric. Tensor  $A_2$  is anisotropic (2nd picture) with 100x more conductivity along the y direction than the x direction (proven above in written work), so there is more conduction along the y direction than the x direction. Note that the eigenvector directions of  $A_2$  are not exactly the x and y axes; the tensor results in a slightly rotated axes, so there is maximal conductivity along one of the eigenvectors (which does not align with the x or y axis). The elliptical shape of conduction in the anisotropic picture would be vertical if the tensor did not have any off diagonal terms. Ultimately, the anisotropy of the 2nd tensor results in uneven conduction along the different axes, while the isotropicity of the 1st tensor results in a radially symmetric conduction.